

Keywords: vision-language-action models • embodied AI • robot manipulation • skill-based planning

EmbodiedSkills: Orchestrating, Training, and Deploying VLA Agents

A Unified Skill Interface with Precondition Checking and Postcondition Verification Lifts Robot Manipulation to 86% Success

By Wei Wang, Wenqiao Zhang, Yutong Lin, et al. (17 authors; Zhejiang University et al.)

arXiv:2609.01281 • Preprint, September 2026

Vision-Language-Action (VLA) models map visual observations and language instructions to robot actions end-to-end, but they struggle with long-horizon tasks because early errors cascade without a recovery mechanism. EmbodiedSkills wraps VLA policies in an executable-skill abstraction that checks prerequisites before execution and verifies outcomes afterward. Task-adapted low-level VLA policies within this framework achieve 86.20% average success across 50 RoboTwin 2.0 tasks and 97.40% across four LIBERO suites — far above frozen-policy baselines.

AT A GLANCE

Problem: VLA models fail on long-horizon tasks because they have no mechanism to detect or recover from execution errors.

Why it matters: Real-world robot deployment requires reliable multi-step manipulation; a single failure can ruin a long task.

Method: Executable-skill interface with learned precondition and postcondition classifiers; modular VLA policy adaptation.

Result: 86.20% average success (RoboTwin 2.0), 97.40% (LIBERO); 62% reduction in cascading failures on 6+ skill tasks.

Evidence: Comparison vs. frozen OpenVLA and task-adapted VLA without skill structure across 54 robot manipulation tasks.

The Big Picture

VLA models have demonstrated impressive generalisation on short-horizon manipulation tasks: given an image observation and a language description, they generate motor commands that successfully complete simple pick-and-place or push tasks. However, real-world applications require long-horizon plans: setting a table, assembling a product, or restocking a shelf involves sequences of 6–10

or more distinct sub-tasks.

In long-horizon settings, VLA models fail because they lack two critical properties:

- **Error detection:** the model has no way to know whether a sub-task succeeded or failed before moving to the next.
- **Recovery:** when a sub-task fails, the model cannot re-try or select an alternative action sequence.

EmbodiedSkills introduces both properties by structuring VLA execution around *executable skills* — a design pattern borrowed from classical task-and-motion planning but implemented entirely with learned neural components.

Why Existing Approaches Fall Short

Monolithic VLA policies (e.g. OpenVLA, RT-2) are trained end-to-end and have no explicit sub-task structure. They may hallucinate task completion or proceed past failures.

Hierarchical RL approaches train separate high-level and low-level policies, but the high-level policy must be retrained for each new task, and low-level sub-policies typically cannot be reused across hierarchies.

Classical task planners (e.g. PDDL-based) offer structured execution with precondition/postcondition semantics but require hand-coded predicates and cannot handle the visual complexity of real-world robot environments.

EmbodiedSkills occupies the gap: learned precondition/postcondition classifiers from visual observations, structured execution via a fixed skill interface, and task-adapted VLA policies as low-level executors.

Core Mechanism

A skill s is a tuple:

$$s = (\text{name}, \text{pre}(o), \pi_s(o, g), \text{post}(o))$$

where:

- $\text{pre}(o) \rightarrow \{0, 1\}$: checks whether the robot is in a valid state to begin executing s .
- $\pi_s(o, g) \rightarrow a$: the low-level VLA policy that generates actions to achieve sub-goal g from observation o .
- $\text{post}(o) \rightarrow \{0, 1\}$: checks whether skill s completed successfully.

The agent loop selects skills via a vision-language model planner, checks preconditions, executes until postcondition is satisfied or a timeout is reached, and triggers recovery or replanning on failure.

Technical Formulation

4.1 High-Level Planning

The high-level planner is a VLM queried at each step as:

$$s_t, a_t = \text{VLM}(G, o_t, \mathcal{S}) \quad (1)$$

where G is the task goal, o_t is the current observation, and $\mathcal{S}$ is the skill library. The VLM reasons in natural language via chain-of-thought before selecting the next skill s_t and its arguments a_t .

4.2 Precondition and Postcondition Classifiers

Pre- and postconditions are binary classifiers trained on annotated demonstrations:

$$f_{\text{pre}}(o; \theta_{\text{pre}}) \rightarrow P(\text{pre satisfied}) \quad (2)$$

$$f_{\text{post}}(o; \theta_{\text{post}}) \rightarrow P(\text{post satisfied}) \quad (3)$$

with positive/negative labels derived from states before and after successful vs. failed skill executions.

4.3 Task-Adapted Policy Training

Each skill’s low-level policy π_s is trained with a combination of behavioural cloning and proximal policy optimisation:

$$\mathcal{L}_s(\theta) = \mathcal{L}_{\text{BC}}(\theta) + \lambda \mathcal{L}_{\text{PPO}}(\theta, R_s)$$

where R_s is a dense reward derived from the postcondition classifier:

$$R_s(o_t) = f_{\text{post}}(o_t; \theta_{\text{post}})$$

This reward is dense — it provides a training signal at every step proportional to how close the current state is to satisfying the postcondition — avoiding the sparse reward problem common in robot RL.

4.4 Recovery Policy

Failure modes are classified by the postcondition checker; each failure mode has an associated recovery policy r selected from a predefined recovery library:

$$r_t = \text{RecoveryPolicy}(o_t, s_t, \text{failure_mode})$$

Learning or Inference Procedure

EmbodiedSkills is trained in three stages:

1. **Precondition/postcondition learning.** Classifiers are trained on annotated state transitions from expert demonstrations.
2. **Low-level VLA adaptation.** Each skill policy is fine-tuned from a pretrained VLA (OpenVLA) using BC + PPO with the postcondition reward. Adaptation is independent per skill.

3. **High-level planner.** A pretrained VLM (7B or 72B) is prompted with the skill library and task specification; no VLM fine-tuning is required.

At inference, the agent loop runs: plan → check precondition → execute → verify postcondition → recover or continue.

What the Guarantee Says

The paper proves that under the learned classifier regime, the expected task completion rate $P(\text{task})$ satisfies:

$$P(\text{task}) \geq \prod_{i=1}^K P(\text{success}_i) \cdot (1 - P(\text{cascade}))$$

where K is the number of skills, $P(\text{success}_i)$ is the per-skill success rate, and $P(\text{cascade})$ is the probability of a cascade failure that the recovery policy cannot resolve. Adding postcondition verification suppresses $P(\text{cascade})$ by catching failures before the next skill begins.

Experimental Findings

Method	RoboTwin 2.0	LIBERO
Frozen VLA (OpenVLA)	51.4%	78.6%
Task-adapted (no framework)	71.2%	89.1%
EmbodiedSkills	86.2%	97.4%

Long-horizon tasks (8+ skills): EmbodiedSkills achieves 74.3% completion vs. 31.2% for the task-adapted baseline without skill structure, a 2.4× improvement attributable to verification-driven recovery.

Ablations and Interpretation

Without postcondition verification: Success drops from 86.2% to 61.4% on RoboTwin 2.0 — the largest single ablation contributor. Cascading failures account for the majority of the 24.8-point gap.

Without precondition checking: Success drops to 74.8%, showing that preventing skill execution from an invalid start state avoids a different class of compounding errors.

VLM planner size: Using a 7B planner vs. a 72B planner reduces success by 4.1% on RoboTwin 2.0. The structured skill interface compensates for planner imperfection: the planner only needs to choose the right skill, not the right low-level action.

Fixed vs. adaptive skill library: A fixed library of 28 primitive skills matches a dynamically grown library for the tested tasks, suggesting that a compact skill vocabulary covers the space of practical manipulation scenarios.

Dense reward (postcondition) vs. sparse reward:

Using the postcondition classifier as a dense reward (vs. sparse 0/1 completion reward) increases average per-skill success rate by 11.4 points.

Reference

Wei Wang, Wenqiao Zhang, Yutong Lin et al. **EmbodiedSkills: A Unified Framework for Orchestrating, Training, and Deploying VLA Agents.** arXiv:2609.01281, September 2026. <https://arxiv.org/abs/2609.01281>